

# EventBus：Android事件发布/订阅框架的使用指南

EventBus 是 Android 中一款轻量级的**事件发布/订阅框架**，核心作用是简化组件间（如 Activity、Fragment、Service 之间）的通信，替代传统的 Intent、Handler、接口回调等方式，让代码更简洁、解耦。

## 一、核心概念

- **事件（Event）**：任意类型的 Java 对象（普通 POJO 类），用于传递数据。
- **发布者（Publisher）**：发布事件的组件（如 Activity/Fragment/Service），通过 `post()` 发送事件。
- **订阅者（Subscriber）**：接收事件的组件，通过注解标记方法，响应事件。
- **线程模式（ThreadMode）**：指定订阅方法运行的线程（核心特性）。

## 二、基本使用步骤

### 1. 集成 EventBus

在项目 `build.gradle`（Module 级）中添加依赖（以最新稳定版为例）：

Code block

```
1 dependencies {
2     // EventBus 核心库
3     implementation 'org.greenrobot:eventbus:3.3.1'
4 }
```

### 2. 定义事件类（Event）

创建普通 Java 类，用于封装需要传递的数据：

Code block

```
1 // 示例：自定义事件（可传递任意数据）
2 public class MessageEvent {
3     private String content;
4     private int code;
5
6     // 构造方法、getter/setter
```

```

7     public MessageEvent(String content, int code) {
8         this.content = content;
9         this.code = code;
10    }
11
12    public String getContent() { return content; }
13    public int getCode() { return code; }
14 }

```

### 3. 订阅者：注册/注销 + 编写订阅方法

**关键：**订阅者需先注册 EventBus，才能接收事件；销毁时注销，避免内存泄漏。

以 Activity 为例：

Code block

```

1  public class MainActivity extends AppCompatActivity {
2
3      @Override
4      protected void onCreate(Bundle savedInstanceState) {
5          super.onCreate(savedInstanceState);
6          setContentView(R.layout.activity_main);
7          // 注册订阅者
8          EventBus.getDefault().register(this);
9      }
10
11     @Override
12     protected void onDestroy() {
13         super.onDestroy();
14         // 注销订阅者
15         EventBus.getDefault().unregister(this);
16     }
17
18     // 订阅方法：通过 @Subscribe 注解标记，指定线程模式
19     @Subscribe(threadMode = ThreadMode.MAIN) // 主线程执行（适合更新UI）
20     public void onMessageEvent(MessageEvent event) {
21         // 处理事件：获取数据、更新UI等
22         String content = event.getContent();
23         int code = event.getCode();
24         Toast.makeText(this, "收到事件：" + content + ", code: " + code,
25             Toast.LENGTH_SHORT).show();
26     }
27 }

```

### 4. 发布者：发布事件

在任意组件（如另一个 Activity/Fragment/Service）中发布事件：

Code block

```
1 // 点击按钮发布事件
2 btnPost.setOnClickListener(v -> {
3     // 构造事件对象
4     MessageEvent event = new MessageEvent("Hello EventBus", 200);
5     // 发布事件（所有已注册的订阅者都会收到）
6     EventBus.getDefault().post(event);
7 });
```

### 三、核心：线程模式（ThreadMode）

@Subscribe 注解的 threadMode 参数决定订阅方法运行的线程，常用值：

| 线程模式                  | 说明                                            |
|-----------------------|-----------------------------------------------|
| ThreadMode.MAIN       | 主线程（UI 线程）执行，适合更新 UI（不能做耗时操作）                 |
| ThreadMode.BACKGROUND | 后台线程执行：若发布线程是主线程，事件在新线程执行；若发布线程是子线程，直接在该线程执行。 |
| ThreadMode.ASYNC      | 始终在新的子线程执行（无论发布线程是啥），适合耗时操作（如网络请求）。           |
| ThreadMode.POSTING    | 默认值，在发布事件的线程执行（发布者线程是啥，订阅方法就在啥线程）。            |

### 四、进阶用法（可选）

#### 1. 粘性事件（Sticky Event）

发布事件后，后续注册的订阅者仍能接收该事件（适合“先发布、后订阅”场景，如 Activity 跳转传值）。

- 发布粘性事件：

Code block

```
1 EventBus.getDefault().postSticky(new MessageEvent("粘性事件", 100));
```

- 订阅粘性事件：

Code block

```
1 @Subscribe(threadMode = ThreadMode.MAIN, sticky = true) // 标记 sticky = true
2 public void onStickyEvent(MessageEvent event) {
3     // 处理粘性事件
4 }
```

- 移除粘性事件：

Code block

```
1 // 移除指定类型的粘性事件
2 EventBus.getDefault().removeStickyEvent(MessageEvent.class);
3 // 移除所有粘性事件
4 EventBus.getDefault().removeAllStickyEvents();
```

## 2. 优先级与取消事件传递

- 优先级：@Subscribe(priority = 10)（数值越大优先级越高，仅在 ThreadMode.POSTING 模式下生效）。
- 取消传递：在订阅方法中调用 EventBus.getDefault().cancelEventDelivery(event)，后续订阅者将无法接收该事件。

## 五、注意事项

1. 注册/注销成对出现：Activity/Fragment 需在 onCreate() / onDestroy() 或 onStart() / onStop() 中注册/注销，避免内存泄漏。
2. 订阅方法规范：
  - 方法必须是 public，无返回值 (void)。
  - 只能有一个参数（事件类型）。
  - 3.0+ 版本需手动添加 @Subscribe 注解（2.x 是基于反射，无需注解）。
3. 避免在耗时线程更新 UI：ThreadMode.BACKGROUND / ASYNC 模式下，不能直接更新 UI，需通过 Handler 切换到主线程。
4. 混淆配置：若开启混淆，需在 proguard-rules.pro 中添加：

```
1  · -keepattributes *Annotation*
2  -keepclassmembers class * {
3      @org.greenrobot.eventbus.Subscribe <methods>;
4  }
5  -keep enum org.greenrobot.eventbus.ThreadMode { *; }
```

## 六、优缺点

- **优点：**解耦组件通信、代码简洁、支持多线程、粘性事件等扩展。
- **缺点：**过度使用会导致事件泛滥，不易追踪（建议按业务分类事件，避免滥用）。

## 总结

EventBus 的核心流程：**定义事件 → 订阅者注册 + 编写订阅方法 → 发布者发布事件 → 订阅者处理事件**。日常开发中，简单的组件通信（如 Fragment 与 Activity、Service 与 Activity）用 EventBus 能大幅简化代码，是 Android 开发中非常实用的工具。

（注：文档部分内容可能由 AI 生成）